

Class 01: MicroCPU 스펙

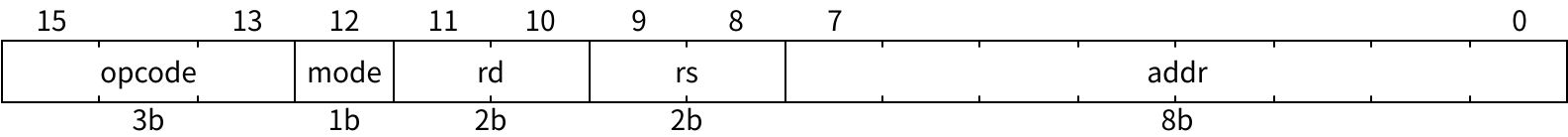
건양대학교 국방반도체공학과
백종섭 교수

- 1.1 MicroCPU 아키텍처 — 16비트 폰 노이만 프로세서의 전체 구조를 설명할 수 있다
- 1.2 MicroCPU 명령어 구조 — 16비트 명령어의 5개 필드를 해석할 수 있다
- 1.3 MicroCPU 명령어 세트 — 주어진 명령어의 동작 결과를 예측할 수 있다
- 1.4 제어 흐름 프로그래밍 — BRZ의 건너뛰기로 if-else, while을 작성할 수 있다
- 1.5 산술/논리 프로그래밍 — 상수 테이블과 NOT+ADD 조합으로 뺄셈, OR 등을 작성할 수 있다

MicroCPU는 학습용으로 설계된, 레지스터·ALU의 워드 폭이 16비트인 폰 노이만 프로세서이다

항목	스펙	설명
워드 폭	16비트	레지스터가 저장하는 실제 값, ALU가 연산하는 실제 값이 모두 16비트이다. R0~R3가 각각 16비트 값을 저장하고, ALU가 16비트 입력을 받아 16비트 결과를 출력한다.
명령어 구조	16비트, 5필드	opcode(3b) + mode(1b) + rd(2b) + rs(2b) + addr(8b). 하나의 명령어 안에 opcode, 주소 지정 방식, 레지스터 2개, 메모리 주소를 인코딩한다.
명령어 세트	3비트 opcode, 8개	제어(WFR), 분기(BRZ, BRA), 데이터 이동(LDA, STA), 산술/논리(ADD, AND, NOT).
메모리	256×16 통합 메모리	256 엔트리, 각 엔트리 16비트 폭. 명령어와 데이터가 동일한 메모리 공간을 공유한다.
레지스터	R0, R1, R2, R3 각 16비트	명령어의 rd, rs 필드가 이 4개 레지스터 중 하나를 선택한다. Rd는 ALU의 첫 번째 입력이면서 결과가 저장되는 대상이다. Rs는 레지스터 모드(mode=1)에서 ALU의 두 번째 입력을 제공한다.
주소 지정	메모리, 레지스터	ADD, AND, LDA opcode만 주소 지정 방식으로 메모리와 레지스터 입력을 선택할 수 있다.
클럭	단일 클럭	모든 내부 블록이 단일 클럭으로 동작한다. WFR 명령어 실행 시 클럭이 정지한다.
명령어 주기	8 클럭/명령어	Fetch (3 클럭): 메모리에서 명령어를 읽는다. Decode (1 클럭): 명령어를 필드별로 해석한다. Execute (4 클럭): 피연산자를 접근하고 연산한 뒤 결과를 저장한다.

16비트 명령어를 5개 필드로 나누어 opcode, 주소 지정 방식, 레지스터 2개, 메모리 주소를 인코딩한다



필드	비트	이름	설명
opcode	[15:13]	명령어 코드	8개 명령어를 인코딩한다.
mode	[12]	주소 지정 방식	ALU의 두 번째 입력 소스를 선택한다. mode=0 (메모리 모드): mem[addr]을 ALU에 전달한다. mode=1 (레지스터 모드): Rs=regfile[rs] 값을 ALU에 전달한다.
rd	[11:10]	목적 레지스터 주소	2비트로 R0(00)~R3(11) 중 하나를 선택한다. 선택된 Rd는 ALU의 첫 번째 입력이면서 결과 저장 대상이다.
rs	[9:8]	소스 레지스터 주소	2비트로 R0(00)~R3(11) 중 하나를 선택한다. 선택된 Rs는 레지스터 모드(mode=1)에서 ALU의 두 번째 입력이 된다.
addr	[7:0]	메모리 주소	8비트로 256개 메모리 엔트리 중 하나를 지정한다. 메모리 모드(mode=0)에서 피연산자 주소 또는 분기 주소로 사용된다.

3비트 opcode로 제어, 분기, 데이터 이동, 산술/논리 4개 그룹 8개 명령어를 정의한다

그룹	Opcode	이름	인코딩	동작	설명
제어	WFR	Wait For Reset	000	프로세서 정지	clock gating으로 전체 정지. rst_n으로만 해제
분기	BRZ	Branch if Zero	001	Rd=0이면 PC += 1	유일한 조건 분기. Rd가 0이면 다음 명령어를 건너뛰다
	BRA	Branch Always	010	PC = addr	무조건 분기. addr[7:0]을 PC에 로드
데이터 이동	LDA	Load	011	Rd = mem[addr]	메모리 값을 레지스터에 로드
	STA	Store	100	mem[addr] = Rd	Rd 값을 메모리에 저장
산술/논리	ADD	Add	101	Rd = Rd + mem[addr]	16비트 이진 덧셈
	AND	And	110	Rd = Rd & mem[addr]	16비트 비트 단위 논리곱
	NOT	Not	111	Rd = ~Rd	16비트 비트 반전. Rd만 사용한다

- ADD, AND, LDA는 주소 지정 방식에 따라 mode=0이면 mem[addr], mode=1이면 Rs를 피연산자로 사용한다

BRZ(조건 skip)와 BRA(무조건 분기) 2개 명령어로 모든 제어 흐름을 구현한다. 구현 가능한 프로그래밍 구문은 if-then, if-else, while, for, do-while 등이다

if-else

BRZ는 $R0 == 0$ 이면 건너뛴다. 건너뛰면 THEN, 건너뛰지 않으면 ELSE를 실행한다.

```
if (R0 == 0) R1 = mem[0x80];
else        R1 = mem[0x81];
```

```
0x0F ... // 이전 코드
0x10 BRZ R0 // R0==0이면 0x11 건너뛴
0x11 BRA 0x14 // R0≠0 → ELSE
0x12 LDA R1, mem[0x80] // THEN: R1 = mem[0x80]
0x13 BRA 0x15 // → 합류
0x14 LDA R1, mem[0x81] // ELSE: R1 = mem[0x81]
0x15 ... // 다음 코드
```

if-then

ELSE가 없으면 BRA 없이 BRZ + skip 1줄로 구현한다.

while 루프

BRZ는 $R0 == 0$ 이면 건너뛴다. 건너뛰면 탈출, 건너뛰지 않으면 본문을 실행한다.

```
while (R0 != 0) R0 -= 1;
```

```
0x0F ... // 이전 코드
0x10 BRZ R0 // R0==0이면 0x11 건너뛴 → 탈출
0x11 BRA 0x13 // R0≠0 → 본문
0x12 BRA 0x15 // 탈출
0x13 ADD R0, mem[0x82] // 본문: R0 = R0 + (-1)
0x14 BRA 0x10 // → 조건 검사
0x15 ... // 다음 코드
```

for 루프

카운터 초기화 후 while과 동일한 패턴으로 구현한다.

do-while

본문을 먼저 실행하고 끝에 BRZ + BRA로 반복 여부를 판단한다.

ADD, AND, NOT 3개 명령어와 상수 테이블로 모든 산술/논리 연산을 수행한다

상수 테이블

- 자주 쓰는 상수(0, 1, -1 등)를 메모리에 미리 저장한다.

```
@80
0000000000000000 // 0x80: 상수 0
0000000000000001 // 0x81: 상수 1
1111111111111111 // 0x82: 상수 -1
0000000011111111 // 0x83: 마스크 0x00FF
```

덧셈

- ADD로 메모리 값 또는 Rs를 더한다.
- 메모리 값이 지정된 상수 값이면 지정된 연산을 한다.
 - 예를 들어서 증가(+1), 감소(-1)

```
0x10 ADD R0, R1 // R0 = R0 + R1
0x11 ADD R0, mem[0x84] // R0 = R0 + mem[0x84]
0x12 ADD R0, mem[0x81] // R0 = R0 + 1
0x13 ADD R0, mem[0x82] // R0 = R0 - 1
```

비트 반전

- NOT으로 16비트 전체를 반전한다.

```
0x17 NOT R0 // R0 = ~R0
```

뺄셈 (A - B)

- A + (-B)를 수행하기 위해서 NOT(B) + 1로 -B를 만들고 A와 ADD.

```
0x14 NOT R1 // R1 = ~R1
0x15 ADD R1, mem[0x81] // R1 = ~R1 + 1 = -R1
0x16 ADD R0, R1 // R0 = R0 + (-R1)
```

NOT + AND로 모든 논리 연산

- 논리 OR 는 $A \mid B = \sim(\sim A \& \sim B)$

```
0x17 NOT R0 // R0 = ~R0
0x18 NOT R1 // R1 = ~R1
0x19 AND R0, R1 // R0 = ~R0 & ~R1
0x1A NOT R0 // R0 = ~(~R0 & ~R1) = R0 | R1
```